ECOMMIND · AUTONOMOUS COMMERCE FOR SELLERS

EcomMind architecture

The seller's agent forecasts demand, buys data per call from service agents, negotiates restocks with supplier agents and settles through escrow. It works inside a spending mandate the seller sets, and every rupee passes one spend check. Today the whole system runs in the browser on a seeded demo business.

Agents · Trust and money · App and control

Seller · browser
Signs in, sets the mandate, approves deals, watches agents work

clicks, forms, CSV / XLSX uploads

BROWSER · REACT 19 + VITE SPA · NO BACKEND YET

Routes · main.tsx → App.tsx
HashRouter · RequireAuth / PublicOnly guards · /login /register /onboarding
/inventory /analytics /insights /agents /suppliers /negotiations /activity /wallet /escrow /reports /import /settings

Layout shell · components/layout/
AppSidebar (Workspace · Agent Economy · Finance · System) · AppHeader (search, notifications, demo loader)

App state · store/AppContext.tsx
One context, 18 actions; auth, mandate and demo flag kept in localStorage
loadDemoBusiness · callAgent · saveMandate · setEscrowStep · addActivity

Seed data · demo.ts
5 products, 90 days of sales
3 suppliers, 4 service agents

every payment

EcomMind · the seller's agent · Insights, Negotiations, Escrow pages
08:55 forecast → 09:00 stockout risk → 09:00 RFQ to 3 suppliers → 09:02 counter-offer
09:05 deal at ₹180 + escrow lock → 09:10 courier quotes → 09:15 return risk → 09:18 prices
Every action becomes an ActivityEvent with its reason

callAgent()
₹ per call · signed receipt

RFQ … counter … accept

open deal
lock / release

Service agents
AgentMarketplace.tsx

Demand forecast	₹4
Market price	₹2
Return risk (per pincode)	₹1
Logistics quote	₹1

blocked above the ₹10/call mandate cap

Supplier agents
Negotiations.tsx

A · NovaSource · 5 days	₹198
B · Kedar · 3 days	₹195 → 180
C · Sona · 8 days	₹189

C dropped: misses the 5-day need

counter ₹175 · floor ₹180
300 units · saves ₹4,500

Escrow
Escrow.tsx · 5-step flow

1 Agreement signed
2 Money locked
3 Goods dispatched (BlueDart)
4 Delivery confirmed
5 Payment released

300 × ₹180 = ₹54,000 locked

TRUST AND MONEY

Mandate · Settings.tsx
₹75,000/day · ₹60,000 max order
₹10 max per call · 15% min margin

Spend check · callAgent()
Paused? Over the per-call cap? Low balance? Blocked, with the reason

Wallet · Wallet.tsx
₹5,000 opening balance, signed txns
Escrow lock / release / refund

Activity + notifications
Activity.tsx · ActivityTimeline
Every agent action with its reason

Import data · ImportData.tsx
CSV / XLSX parsed with SheetJS
Products, sales, returns, quotes

Analytics + reports
Analytics.tsx · Reports.tsx
Recharts: sales, forecast, spend

Where it goes next

Four phases, each closed by a gate. Phase 1 needs no backend, so the demo keeps working the whole way.

PHASE 1 · START NOW

Restructure the front end

- Group pages into features/ and move types to a shared zod package
- Pages call query hooks instead of importing demo.ts
- Serve seed data through a mock API; split AppContext

Gate: the demo runs unchanged on the mock API

PHASE 2

Backend foundation

- API with Postgres, seeded from demo.ts
- Real sign-in and sessions
- Products, sales, returns and import endpoints

Gate: a new seller signs up, imports a CSV and sees their own inventory

PHASE 3

Agents, ledger and escrow

- Mandate check moves to the server, with an approval queue
- Wallet ledger and escrow with refunds
- Live activity feed over server-sent events

Gate: every spend is checked and written to an audit log

PHASE 4

Real integrations

- Sync sales channels
- Real service agents, suppliers and couriers
- Payment partner for top-ups and escrow

Gate: payment partner approved for live funds

React 19 · TypeScript 5.9 · Vite 7 · Tailwind 3 + shadcn/ui · react-router 7 · Recharts · zod · SheetJS

Simulated today: agent responses, receipt signatures, wallet top-ups and escrow movements all run in the browser from seed data. No money moves and no outside service is called.